Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

INŻYNIERIA MECHATRONICZNA | I stopień | semestr VI | 2025/2026

Programowanie Obiektowe i Inżynieria Oprogramowania

Instrukcja C++/CLI: 1. Wprowadzenie

Wprowadzenie do aplikacji okienkowych w C++

Nauczysz się:
Tworzyć szkielet aplikacji okienkowej w języku C++/CLI z wykorzystaniem platformy *.NET framework* w środowisku *Visual Studio*:

- skonfigurujesz środowisko Visual Studio,
- dodasz do projektu pierwsze okna aplikacji (*formsy*),
- utworzysz *menu* aplikacji,
- uruchomisz swoją pierwszą aplikację okienkową.

Dodatkowe materiały:

- materiały do zajęć: [link],
- oficjalna strona C++/CLI [link].

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
IME1A_U1, IME1A_U4

Koordynator przedmiotu:
dr inż. Lucjan Miękina, miekina@agh.edu.pl

Autor instrukcji:
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

1. Informacje wstępne

W serii instrukcji „C++/CLI” zostaną Ci przedstawione podstawy tworzenia aplikacji okienkowych w języku C++/CLI z wykorzystaniem platformy *.NET framework* w środowisku *Visual Studio*. Do tego celu posłużymy się przykładem projektu *kubeczków*, przy czym nie będziemy w pełni implementować rozwiązania – skupimy się na przedstawieniu nowych, istotnych narzędzi, które są specyficzne dla języka C++/CLI i aplikacji okienkowych.

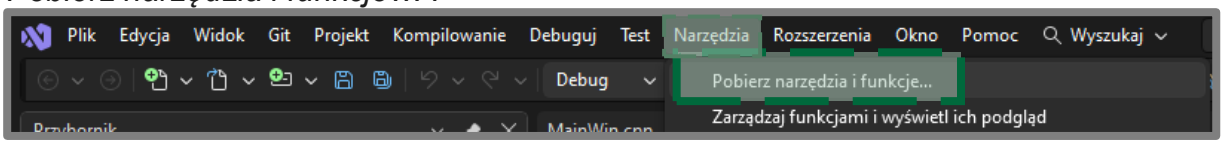
2. Część teoretyczna

C++/CLI to język oprogramowania oparty na C++, stworzony przez firmę *Microsoft*. Aplikacje okienkowe nie są więc tworzone w czystym C++, szczególnie dotyczy to fragmentów kodu związanych z GUI. Niemniej, część logiczna dalej implementowana jest w składni typowej dla C++.

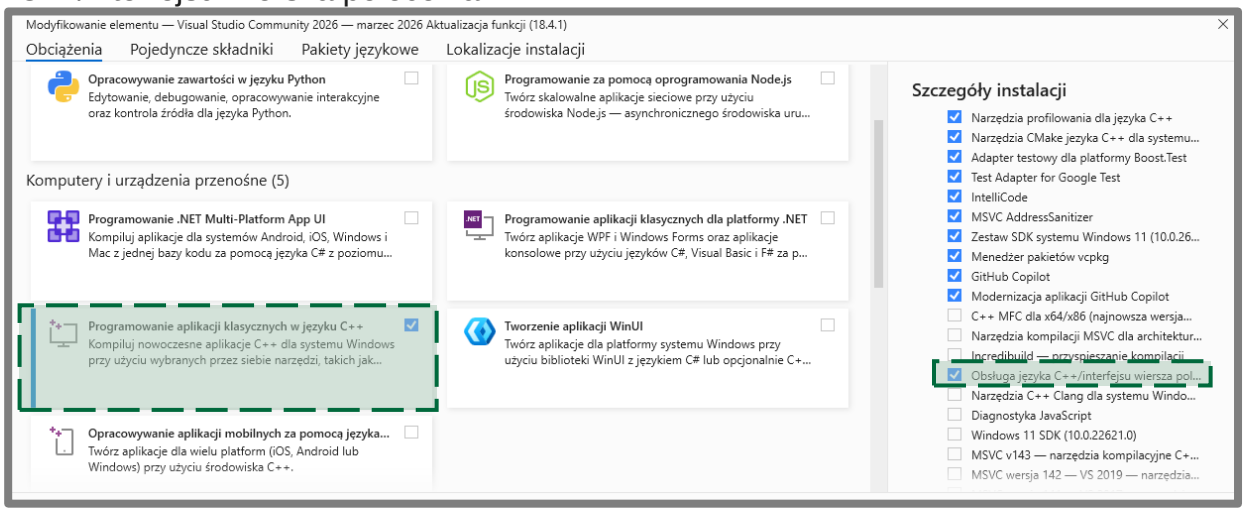
Zastosowanie C++/CLI wydaje się być najprostszą, darmową metodą do nauki tworzenia aplikacji okienkowych na systemy Windows – przy założeniu znajomości języka C++. W podsumowaniu tej części kursu przedstawię Ci alternatywne podejścia.

3. Wprowadzenie

W celu korzystania z C++/CLI w środowisku Visual Studio należy zainstalować dodatkowe komponenty. W otworzonym IDE należy przejść do zakładki *Narzędzia* → *Pobierz narzędzia i funkcje…* .



W otwartym oknie należy wybrać zestaw narzędzi „Programowanie aplikacji klasycznych w języku C++” i w „Szczegóły instalacji” zaznaczyć opcję „Obsługa języka C++/interfejsu wiersza polecenia”:



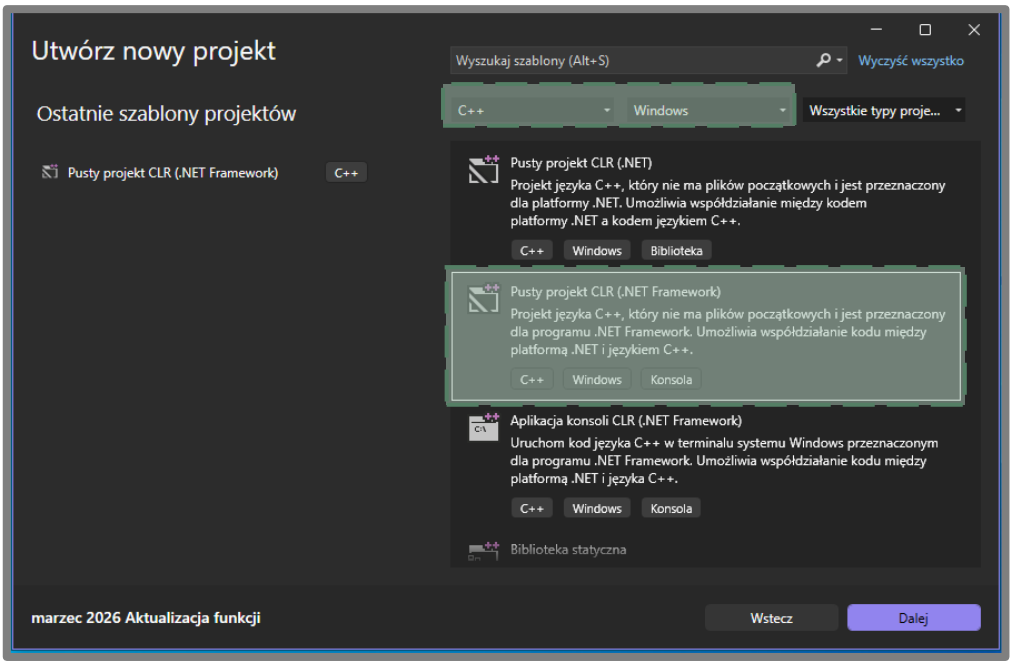
Po pomyślnej instalacji środowisko Visual Studio jest gotowe do pracy.

4. Zadania

Zadania przeprowadzą Cię przez utworzenie pierwszej aplikacji okienkowej w języku C++/CLI. Póki co utworzymy jedynie główne okno programu z wstępnie dodanym menu głównym i dodatkowym oknem wyświetlającym informacje o autorze.

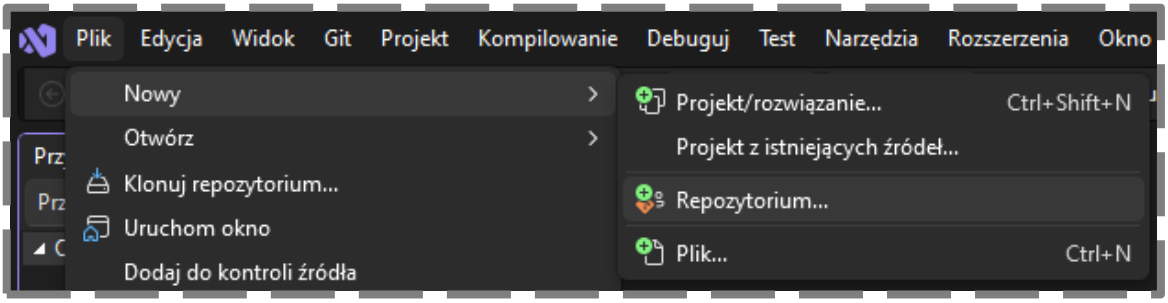
ZADANIE 4.0 – Utworzenie projektu z repozytoriami

Stwórz nowy projekt. Dla języka „C++" i platformy „Windows" odszukaj szablon „Pusty projekt CLR (.NET Framework)":

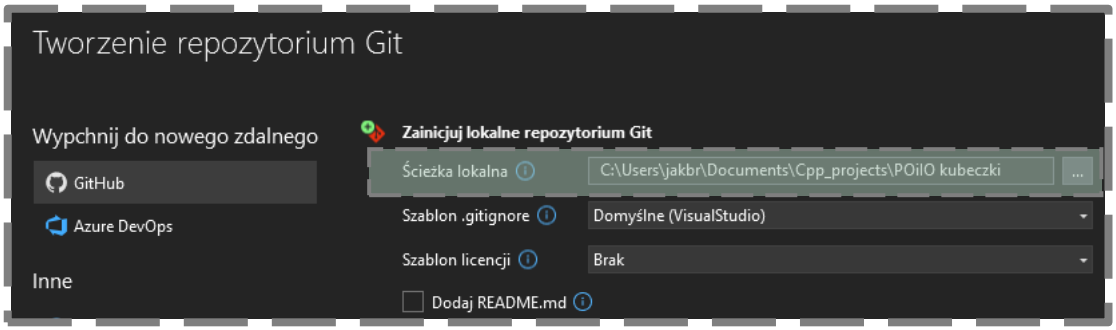


Kliknij „*Dalej*". W nowym oknie nadaj nazwę projektowi, np. „POiIO kubeczki", określ lokalizację projektu i kliknij „Utwórz".

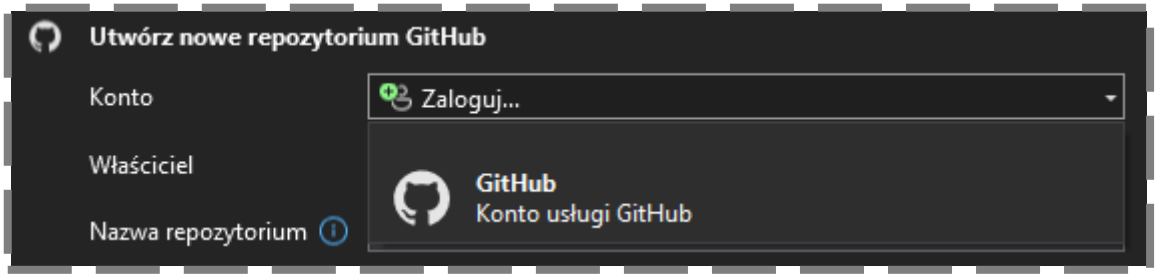
Następnie, przejdziemy do utworzenia repozytoriów. W głównym menu IDE wybierz Plik → Nowy → Repozytorium…



W nowo otwartym oknie wskaż lokalizację dla repozytorium lokalnego – **powinien to być folder projektu Visual Studio**. Zwróć uwagę na opcję „*Szablon .gitignore*” – omówimy ją sobie za chwilę.



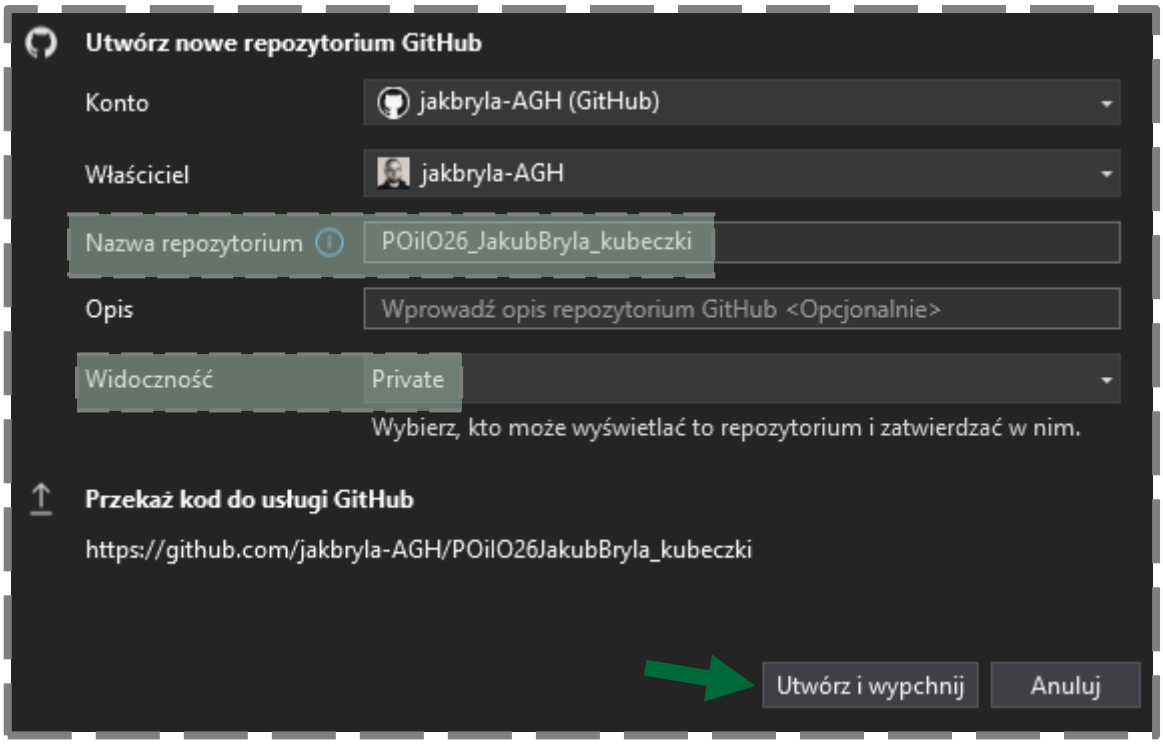
Następnie zaloguj się na swoje konto GitHub:



Po pomyślnej akcji logowania wpisz nazwę repozytorium zdalnego według poniższego klucza, bez polskich znaków:

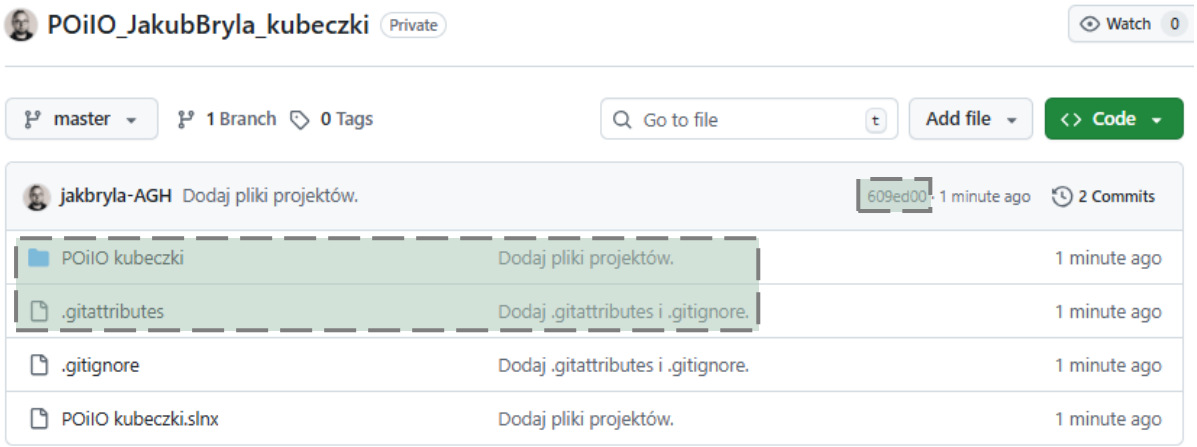
POiIO26_<imie><nazwisko>_kubeczki

Pozostaw repozytorium jako prywatne:

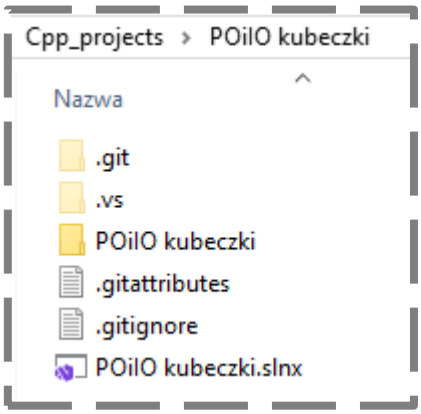


Kliknij przycisk „Utwórz i wypchnij”.

Po kilku chwilach repozytoria zostaną utworzone automatycznie. Przejdź na stronę swojego konta na stronie GitHub i zwróć uwagę na nowe repozytorium projektu – został przesłany *commit* (609ed00), a repozytorium zawiera pliki projektu, *.gitattributes* oraz *.gitignore*:



Sprawdź zwartość folderu w Eksploratorze plików:



Otwórz konsolę cmd, przejdź do lokalizacji repozytorium lokalnego i wpisz komendy:

- git status,
- git branch –vv,
- git log –oneline.

```
C:\Users\jakbr\source\repos\POiIO_kubeczki>cd C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki

C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki>git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki>git branch -vv
* master 609ed00 [origin/master] Dodaj pliki projektów.

C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki>git log --oneline
609ed00 (HEAD -> master, origin/master, origin/HEAD) Dodaj pliki projektów.
5f31286 Dodaj .gitattributes i .gitignore.
```

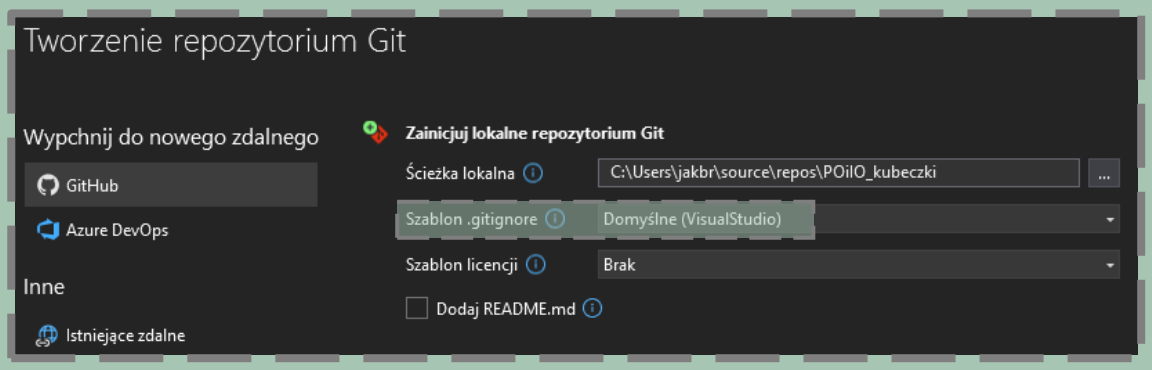
Visual Studio zadbał za nas o:

- utworzenie repozytorium lokalnego i zdalnego,
- utworzenie gałęzi *master* na obu repozytoriach i powiązanie ich ze sobą,
- utworzenie plików konfiguracyjnych i przesłanie ich na repozytorium zdalne – pierwszy commit (5f31286),
- przesłanie plików projektu – drugi commit (609ed00).

W pliku *.gitignore* zamieszcza się nazwy plików i folderów, które mają być nieśledzone przez git – dzięki temu w odpowiedzi instrukcji „git status" nie będą pojawiać się informacje o niedodaniu wybranych plików. Otwórz plik *.gitignore* i zwróć uwagę na notacje – dla przykładowych rekordów z pliku:

- ScaffoldingReadMe.txt → nie śledź pliku „ScaffoldingReadMe.txt"
- *.rsuser → nie śledź żadnego pliku o rozszerzeniu „rsuser",
- mono_crash.* → nie śledź żadnego pliku o nazwie „mono_crash", bez względu na format pliku,
- x64/ → nie śledź zawartości folderu „x64",
- [Dd]ebug/ → nie śledź zawartości folderów: „Debug" i „debug".

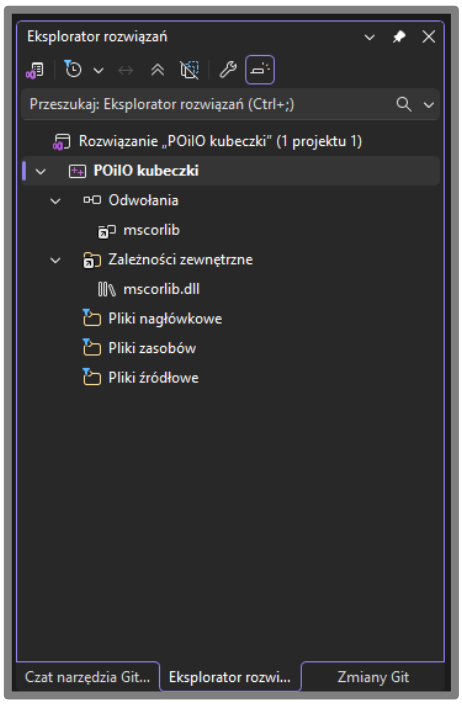
Zawartość pliku *.gitignore* została wygenerowana automatycznie przez Visual Studio przez wybór opcji „*Domyślne (VisualStudio)*" podczas tworzenia repozytoriów:



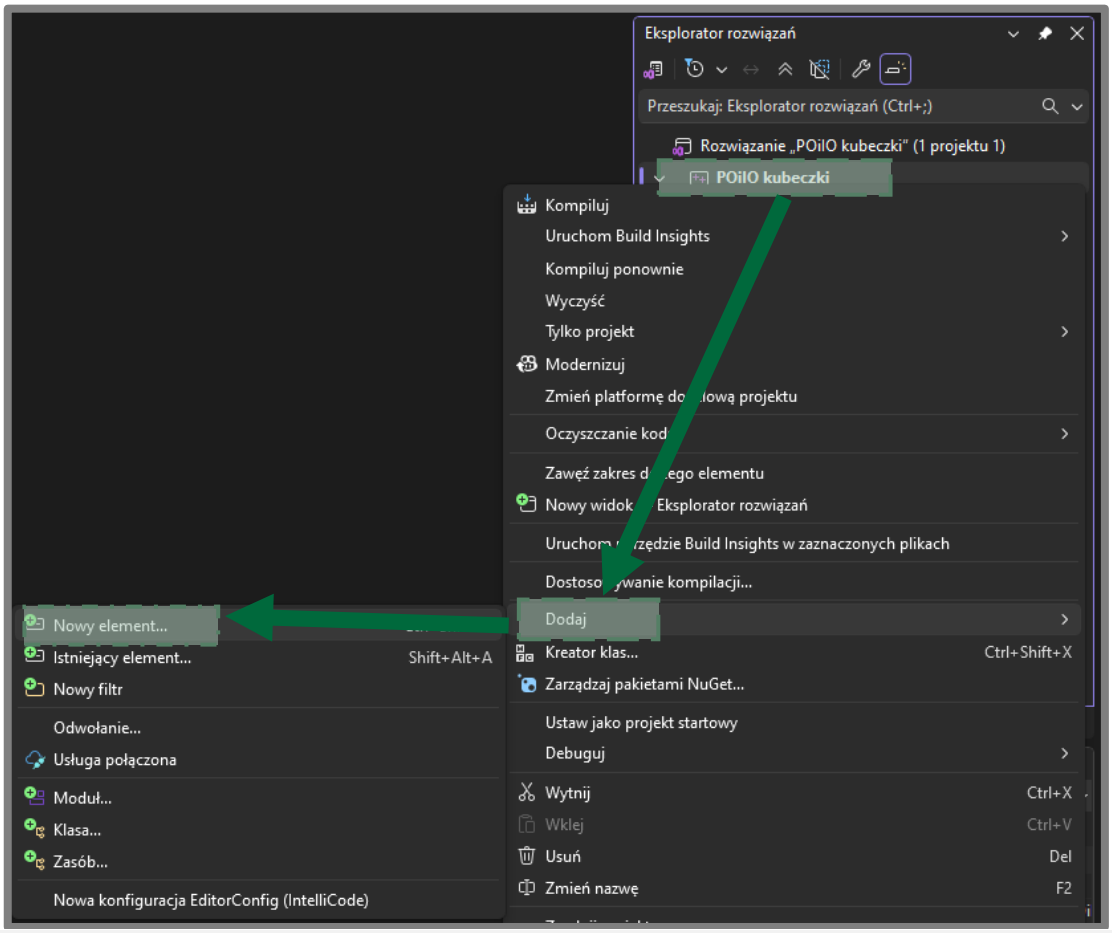
Tym samym Visual Studio zadbał o to, aby do repozytorium zdalnego nie były przesyłane pliki związane z lokalną pracą IDE oraz kompilacją. Nie ma potrzeby przesyłania plików wykonawczych – zajmują one dodatkowe miejsce, a i tak zostaną wygenerowane ponownie podczas kompilacji na innym stanowisku, na którym pobrano aktualny stan repozytorium.

ZADANIE 4.1 – Utworzenie projektu, pierwsze uruchomienie aplikacji

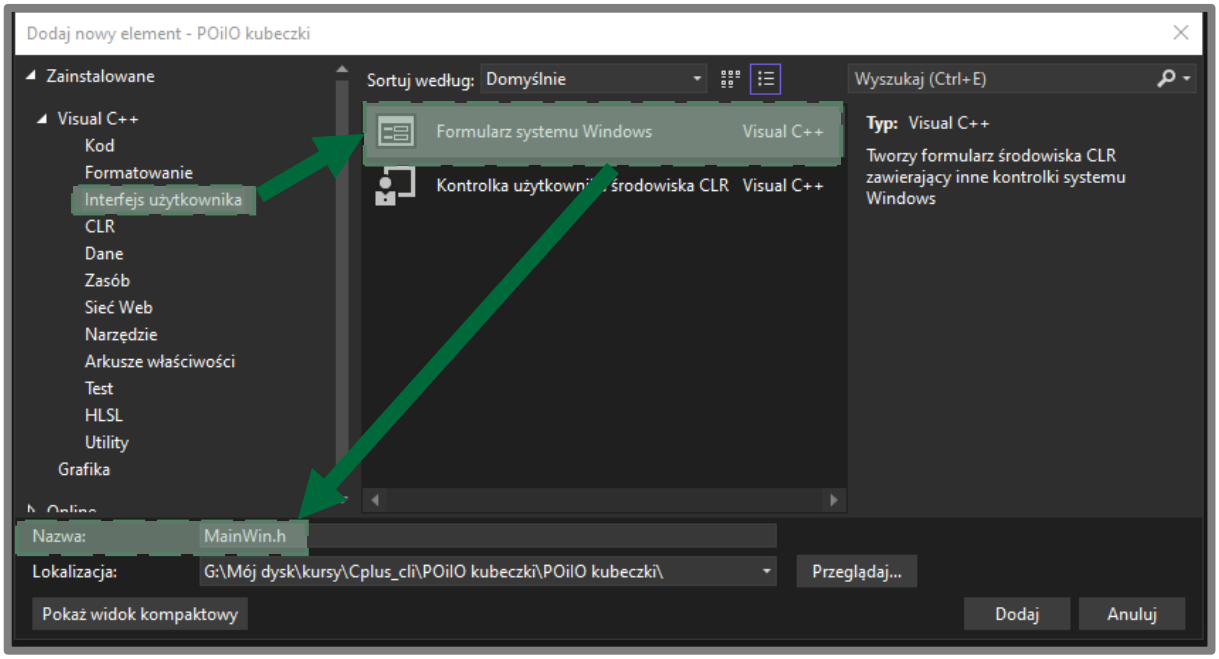
Wracamy do pracy nad aplikacją. Zwróć uwagę na zawartość nowo utworzonego projektu – dołączona jest jedynie jedna biblioteka, projekt jest właściwie pusty:



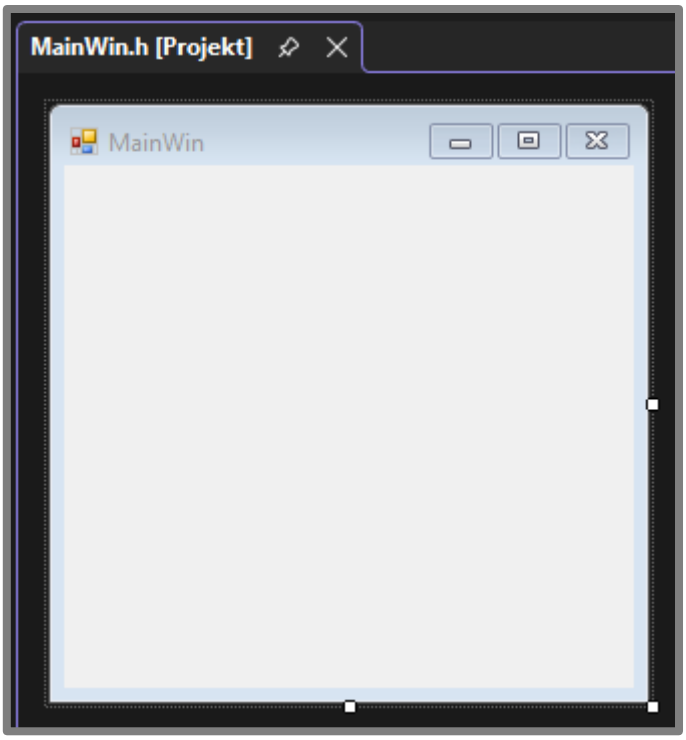
Stwórzmy główne okno programu. Kliknij prawym przyciskiem myszy na nazwę projektu w *Eksploratorze rozwiązań*, a następnie *Dodaj* → *Nowy element…*



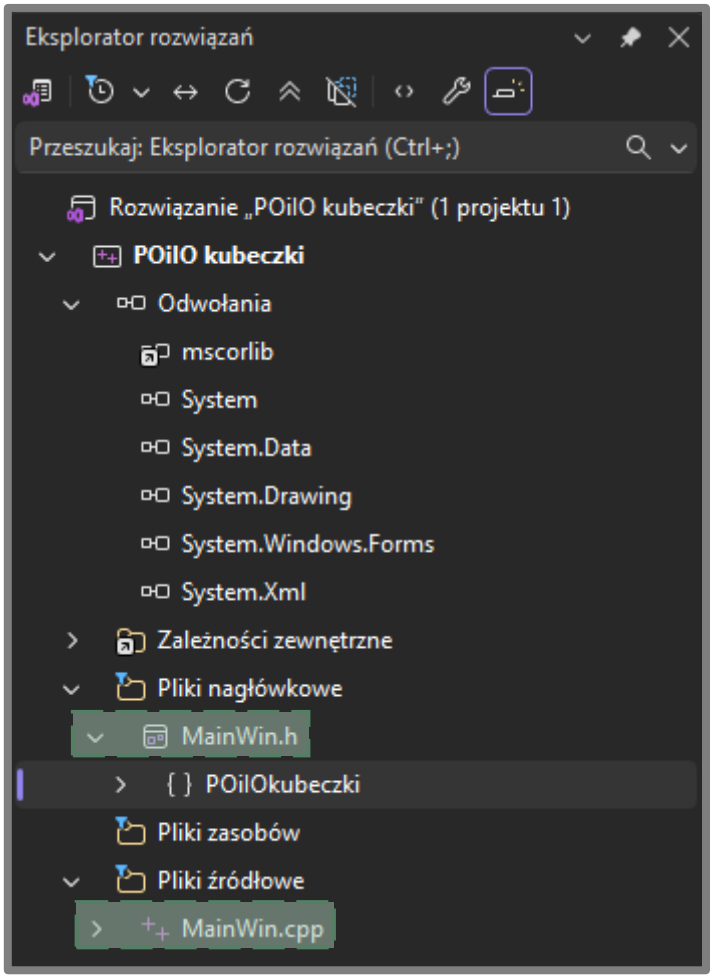
W otwartym oknie wybierz „Interfejs użytkownika”, następnie „Formularz systemu Windows” i nadaj mu nazwę „MainWin.h”:



W głównym widoku IDE Visual Studio ukaże się główne okno aplikacji:



W *Eksploratorze rozwiązań* zobaczysz nowe odwołania i, przede wszystkim, dołączone pliki „MainWin.h” oraz „MainWin.cpp”:



Klikając dwukrotnie lewym przyciskiem myszy na „{} POiIOkubeczki”, wewnątrz *MainWin.h*, otworzy się plik z kodem odpowiedzialnym za utworzenie okna aplikacji (zapewne, w zależności od wersji IDE i *.NET framework*, kod może się nieznacznie różnić):

```
1  #pragma once
2
3  namespace POiIOkubeczki {
4
5          using namespace System;
6          using namespace System::ComponentModel;
7          using namespace System::Collections;
8          using namespace System::Windows::Forms;
9          using namespace System::Data;
10         using namespace System::Drawing;
11
12         /// <summary>
13         /// Podsumowanie informacji o MainWin
14         /// </summary>
15         public ref class MainWin : public System::Windows::Forms::Form
16         {
17         public:
18                 MainWin(void)
19                 {
20                         InitializeComponent();
21                         //
22                         //TODO: W tym miejscu dodaj kod konstruktora
23                         //
24                 }
```

```
25
26          protected:
27                  /// <summary>
28                  /// Wyczyść wszystkie używane zasoby.
29                  /// </summary>
30                  ~MainWin()
31                  {
32                          if (components)
33                          {
34                                  delete components;
35                          }
36                  }
37
38          private:
39                  /// <summary>
40                  /// Wymagana zmienna projektanta.
41                  /// </summary>
42                  System::ComponentModel::Container ^components;
43
44  #pragma region Windows Form Designer generated code
45                  /// <summary>
46                  /// Metoda wymagana do obsługi projektanta - nie należy
    modyfikować
47                  /// jej zawartości w edytorze kodu.
48                  /// </summary>
49                  void InitializeComponent(void)
50                  {
51                          this->components = gcnew
    System::ComponentModel::Container();
52                          this->Size = System::Drawing::Size(300,300);
53                          this->Text = L"MainWin";
54                          this->Padding = System::Windows::Forms::Padding(0);
55                          this->AutoScaleMode =
    System::Windows::Forms::AutoScaleMode::Font;
56                  }
57  #pragma endregion
58          };
59  }
60
```

Nie ma potrzeby dogłębnie analizować powyższy kod. Warto jednak wiedzieć, że w tym pliku zawarte są wszelkie informacje (kod) o tworzonym GUI okna (formy) aplikacji.

Plik *MainWin.cpp* zawiera jedynie odwołanie do pliku nagłówkowego:

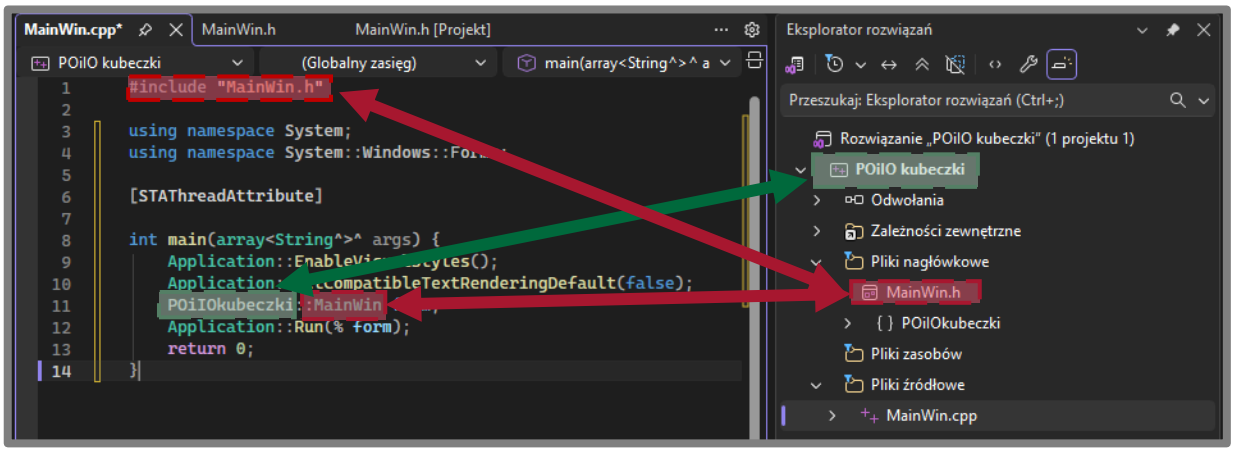
```
1  #include "MainWin.h"
```

W celu poprawnego uruchomienia aplikacji należy uzupełnić plik o poniższe linijki kodu:

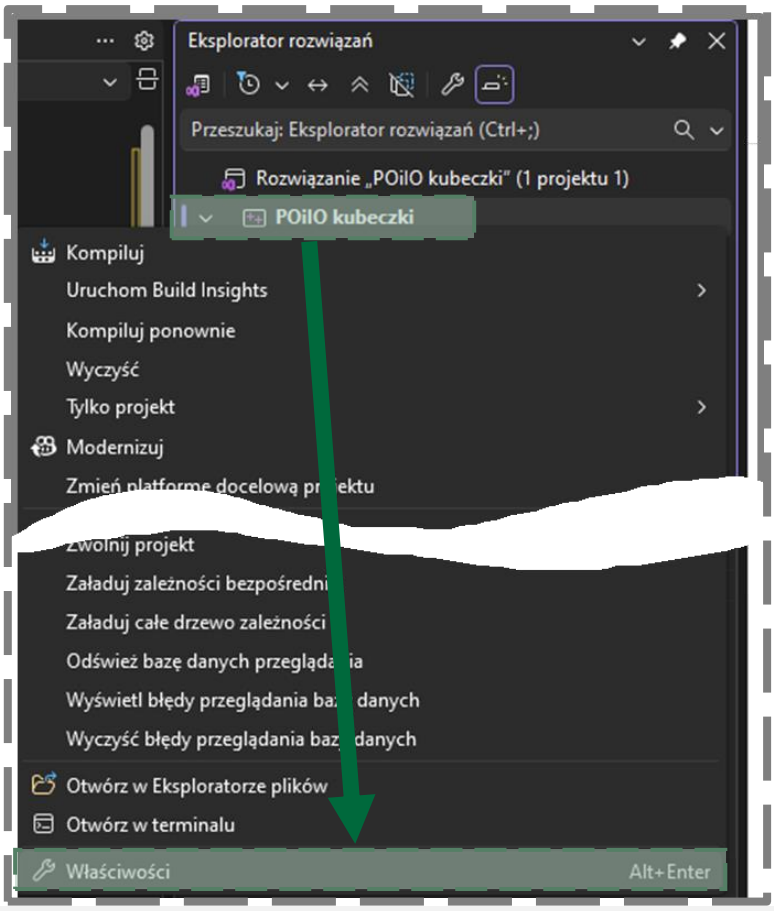
```
1  #include "MainWin.h"
2
3  using namespace System;
4  using namespace System::Windows::Forms;
5
6  [STAThreadAttribute]
7
8  int main(array<String^>^ args) {
9          Application::EnableVisualStyles();
10         Application::SetCompatibleTextRenderingDefault(false);
11         POiIOkubeczki::MainWin form;
12         Application::Run(% form);
13         return 0;
14 }
```

Powyższy kod jest uniwersalny dla każdej tworzonej aplikacji, przy czym dla konkretnej aplikacji należy dokonać następujących modyfikacji:

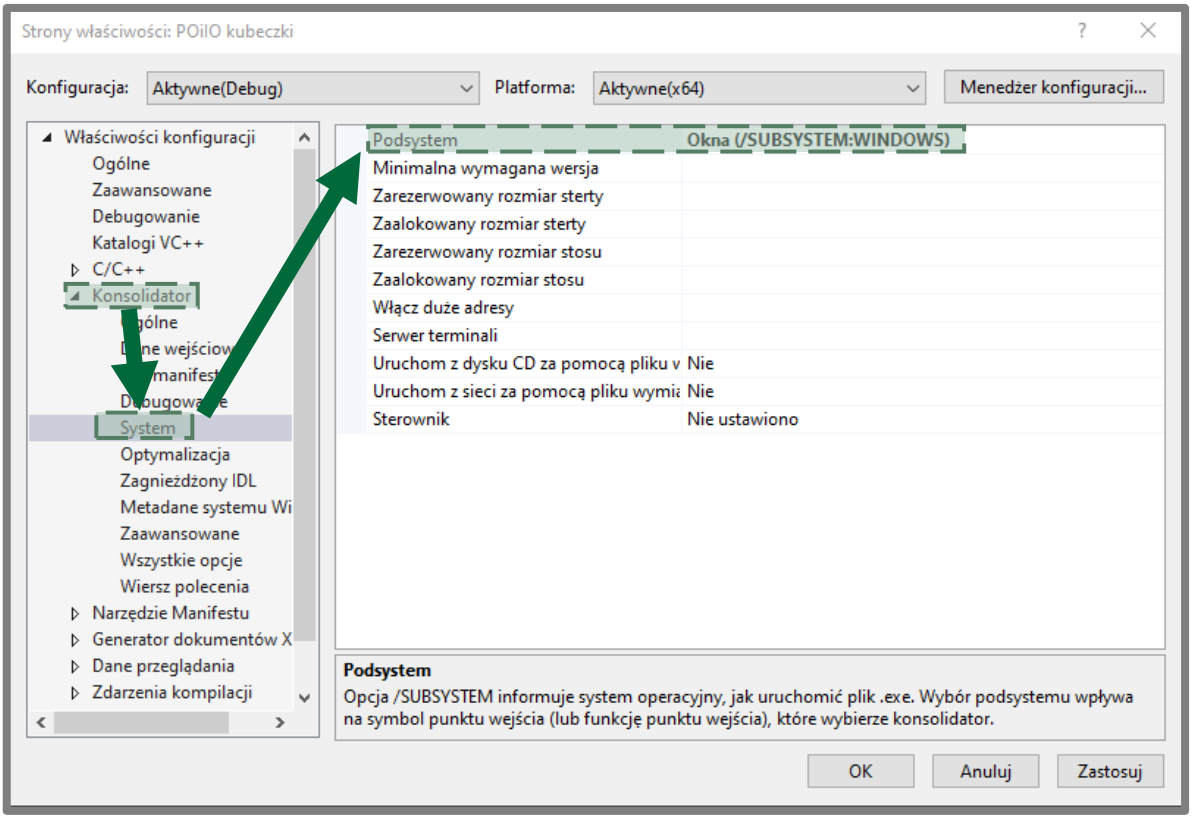
- W linijce 1. odwołać się do odpowiedniego pliku nagłówkowego, w tym przypadku „MinWin.h”,
- W linijce 11. nazwa przestrzeni nazw jest jednocześnie nazwą projektu,
- Tworzony (linijka 11) i uruchamiany (linijka 12) jest obiekt klasy form – utworzonego okna, należy podać jego nazwę, w tym przypadku: „MainWin”.



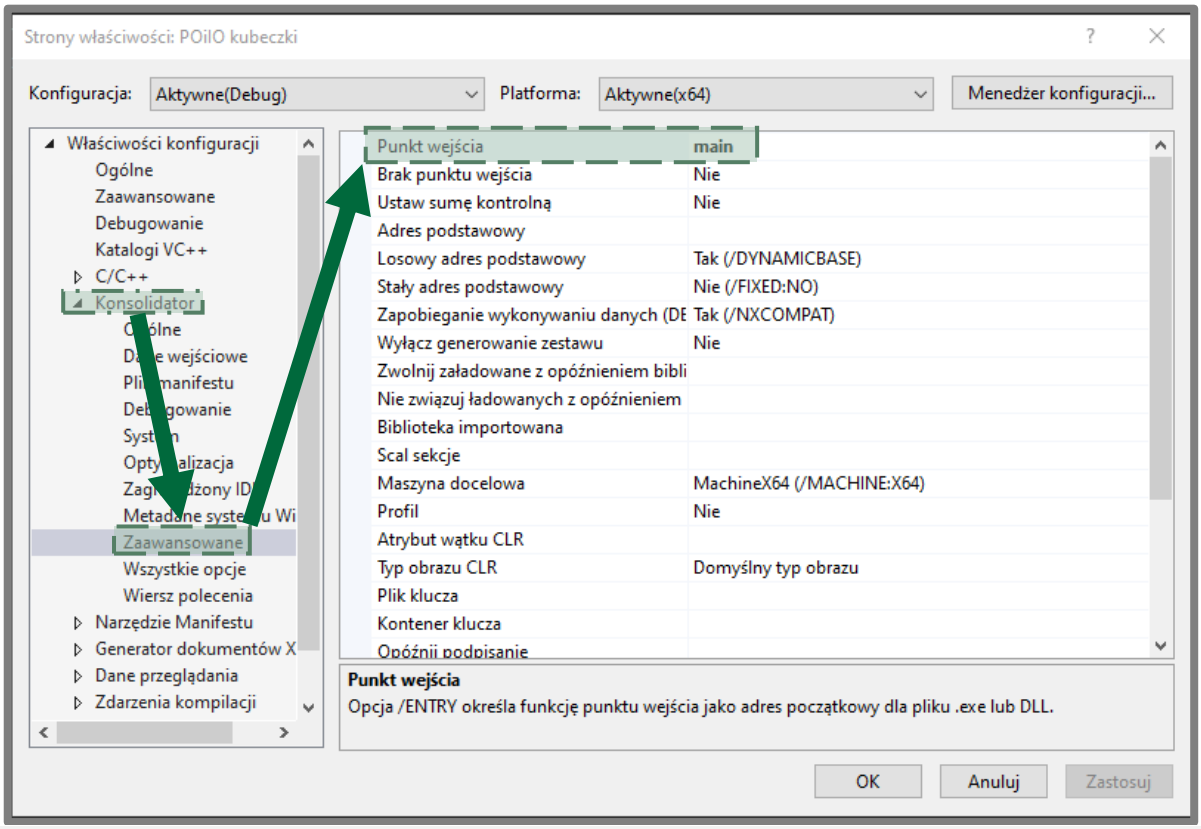
Do pierwszego uruchomienia aplikacji brakuje jedynie ustawienia dwóch parametrów. W tym celu przejdź do ustawień projektu – kliknij prawym przyciskiem myszy w *Eksploratorze rozwiązań* na nazwę projektu i wybierz opcję *Właściwości*:

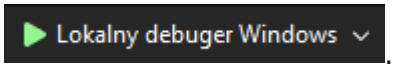


W nowo otwartym oknie wybierz zakładkę „Konsolidator”, a następnie „System”. Dla opcji „Podsystem” wybierz „Okna (…)”:

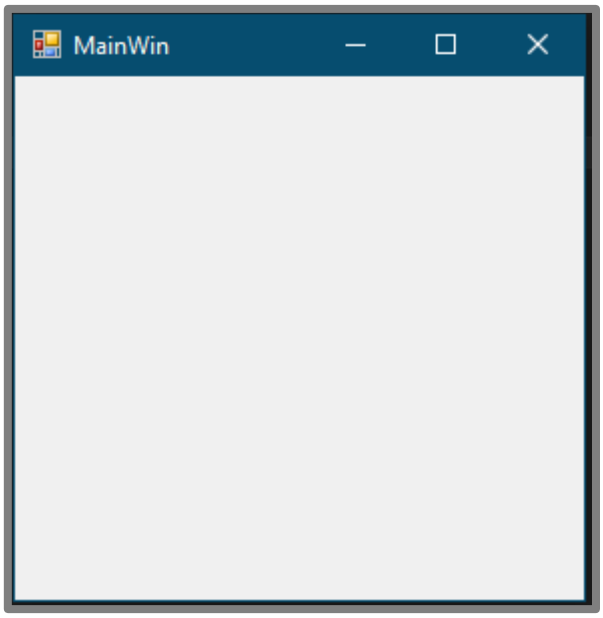


Następnie, w tej samej zakładce „Konsolidator”, przejdź do „Zaawansowane” i dla opcji punkt wyjścia wstaw „main” (lub odpowiednią inną nazwę funkcji, w której uruchamiane jest główne okno aplikacji):

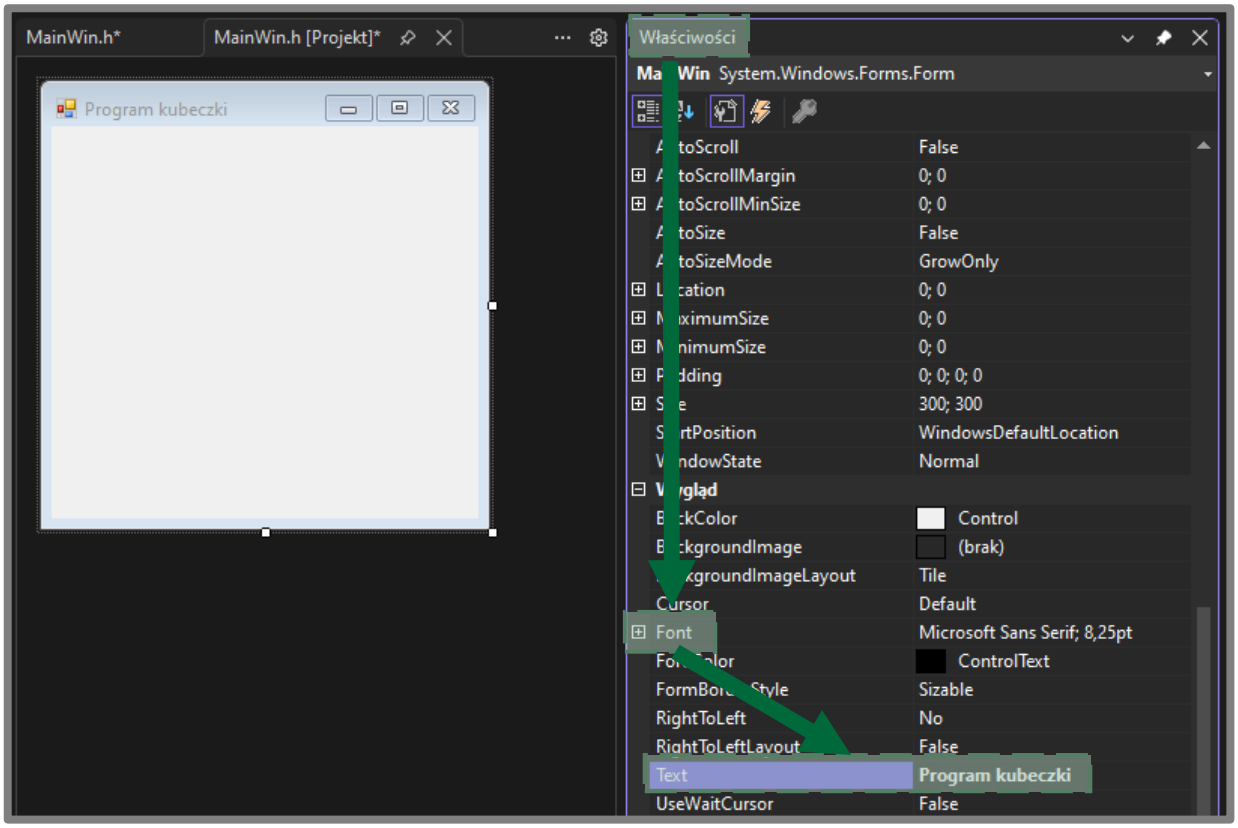


Po zamknięciu okna możesz uruchomić aplikację 

. W efekcie uruchomi się puste okno aplikacji:



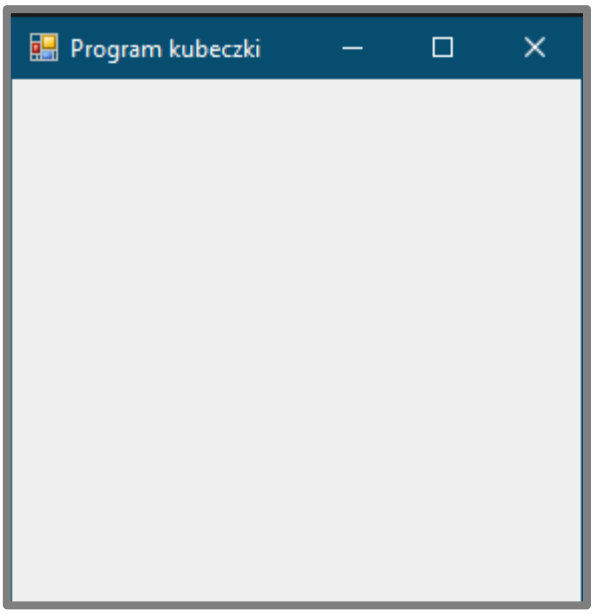
Dla, jeszcze pustego okna aplikacji, możemy zmienić nazwę okna. W tym celu przejdź w głównym widoku IDE do zakładki „MainWin.h [Projekt]”, kliknij raz prawym przyciskiem na okno aplikacji (aktywuj okno) i przejdź do widoku „Właściwości” w prawym dolnym rogu IDE (jeśli jest niewidoczne naciśnij przycisk F4). Odnajdź zakładkę „Font” i opcję „Text” – zmień wartość z „MainWin” na „Program kubeczki”:



Zwróć uwagę, że w pliku „MainWin.h” zaszły automatyczne zmiany wartości parametru:

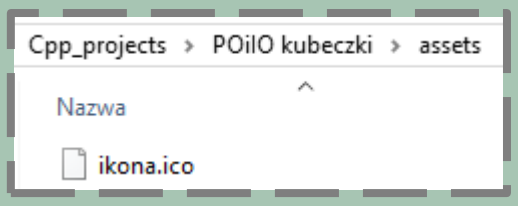
```
44      #pragma region Windows Form Designer generated code
45              /// <summary>
46              /// Metoda wymagana do obsługi projektanta — nie należy modyfikować
47              /// jej zawartości w edytorze kodu.
48              /// </summary>
49              void InitializeComponent(void)
50              {
51                  this->SuspendLayout();
52                  //
53                  // MainWin
54                  //
55                  this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
56                  this->AutoScaleMode = System::Windows::Forms::AutoScaleMode::Font;
57                  this->ClientSize = System::Drawing::Size(284, 261);
58                  this->Name = L"MainWin";
59                  this->Text = L"Program kubeczki";
60                  this->ResumeLayout(false);
61
62              }
63      #pragma endregion
```

Skompiluj i uruchom program:

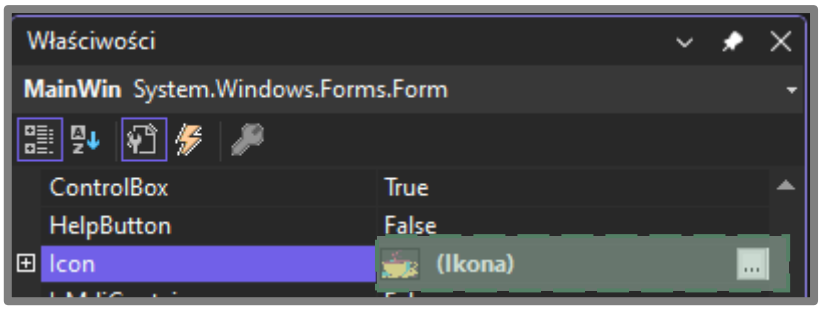


Na koniec możemy zmienić ikonę aplikacji. W tym celu skorzystałem z grafiki dostępnej open-source: link – plik należy zapisać w formacie **.ico*. Plik w odpowiednim formacie został Ci udostępniony na repozytorium wraz z instrukcją.

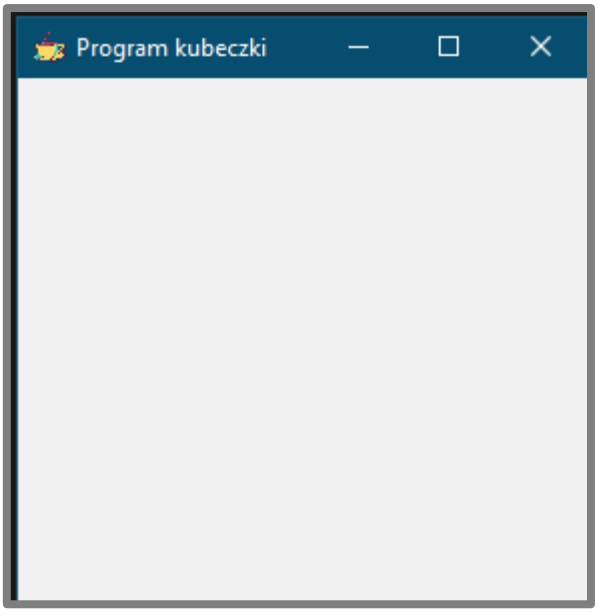
Polecam Ci zapisać plik w folderze projektu, w podfolderze „assets”.



Po zapisaniu pliku możemy go wczytać do projektu:

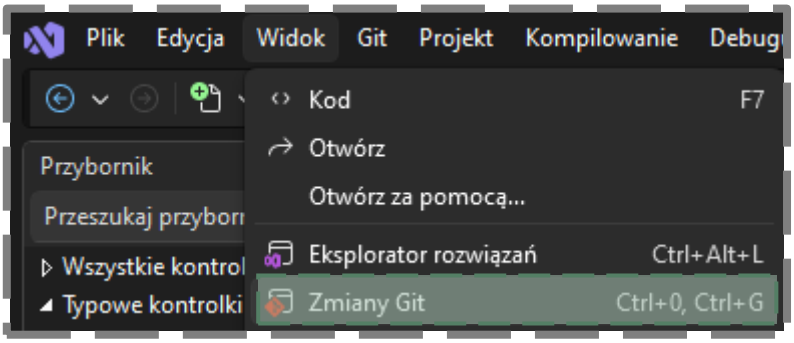


Po kompilacji i uruchomieniu programu otrzymujemy finalny efekt z pierwszego zadania:

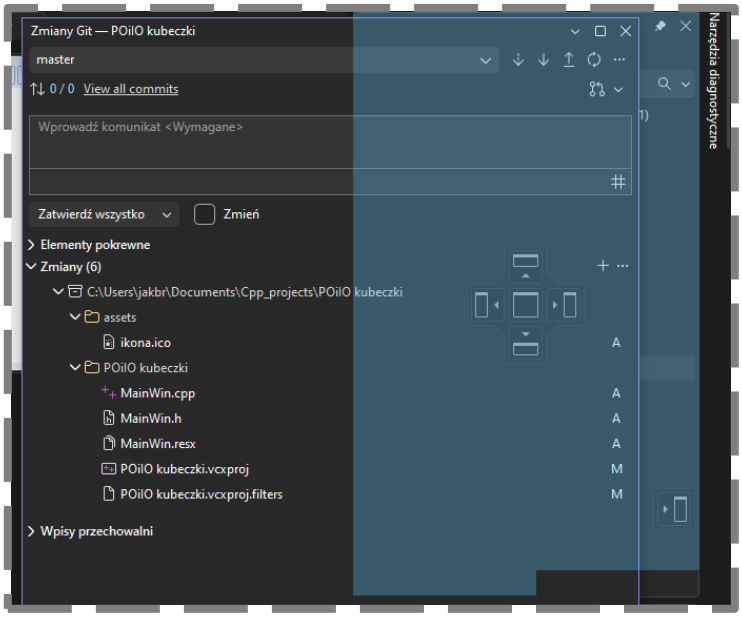


ZADANIE 4.1* – Zapisanie zmian w repozytorium zdalnym

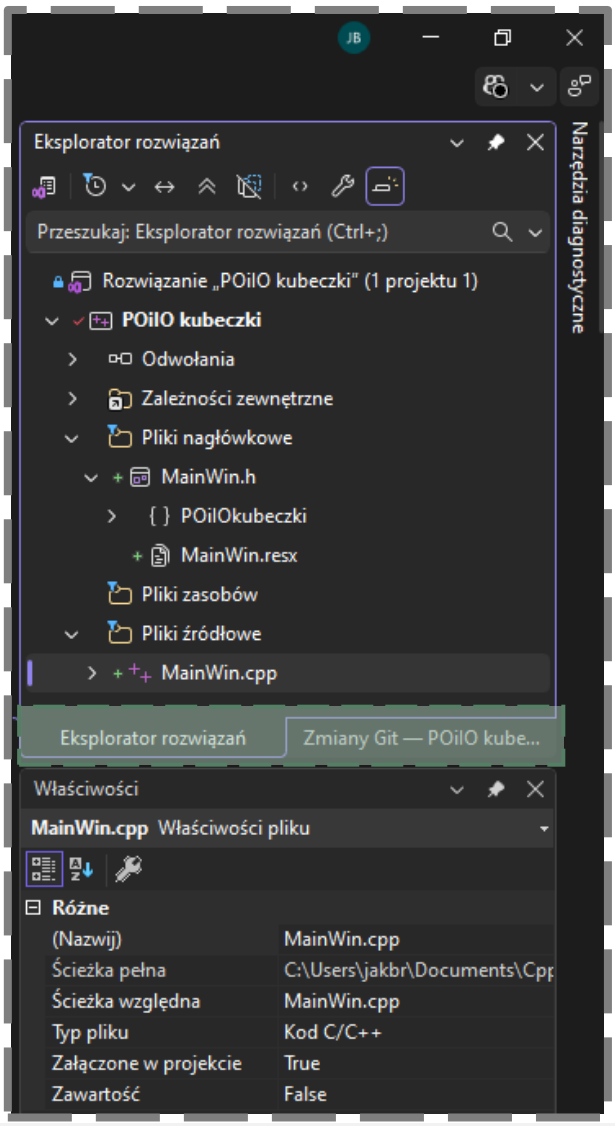
Pozostaje wysłać zmiany na repozytorium zdalne. W pierwszej kolejności otwórz okno „Zmiany Git”.



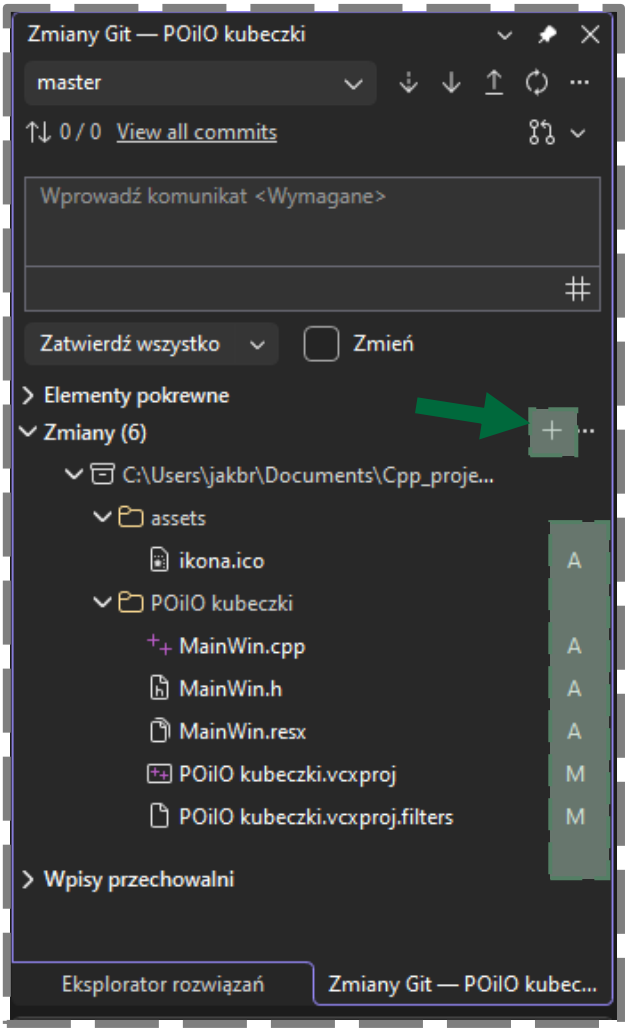
Jeśli nowa zakładka otworzy Ci się w głównym oknie programu, proponuję ją przeciągnąć do „Eksplorator rozwiązań”:



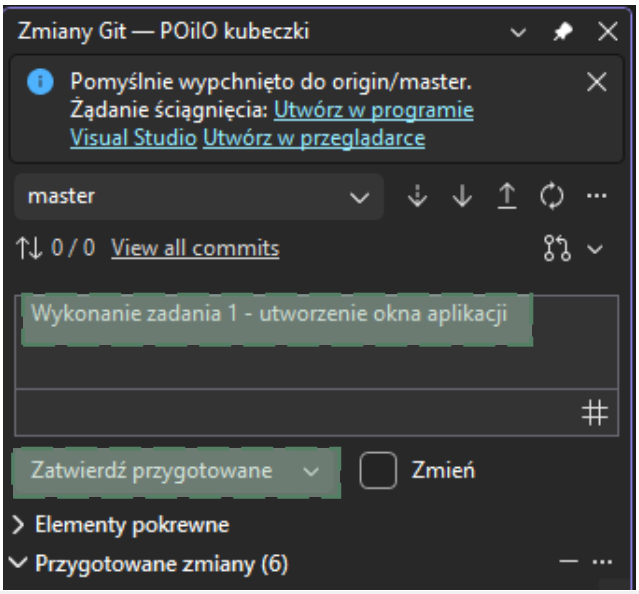
Finalny widok bocznego paska IDE:



Przejdź do zakładki „*Zmiany Git*”. Literki „A” i „M” obok nazw plików oznaczają odpowiednio, że jest to nowy plik lub zmodyfikowany plik względem ostatniego commitu.



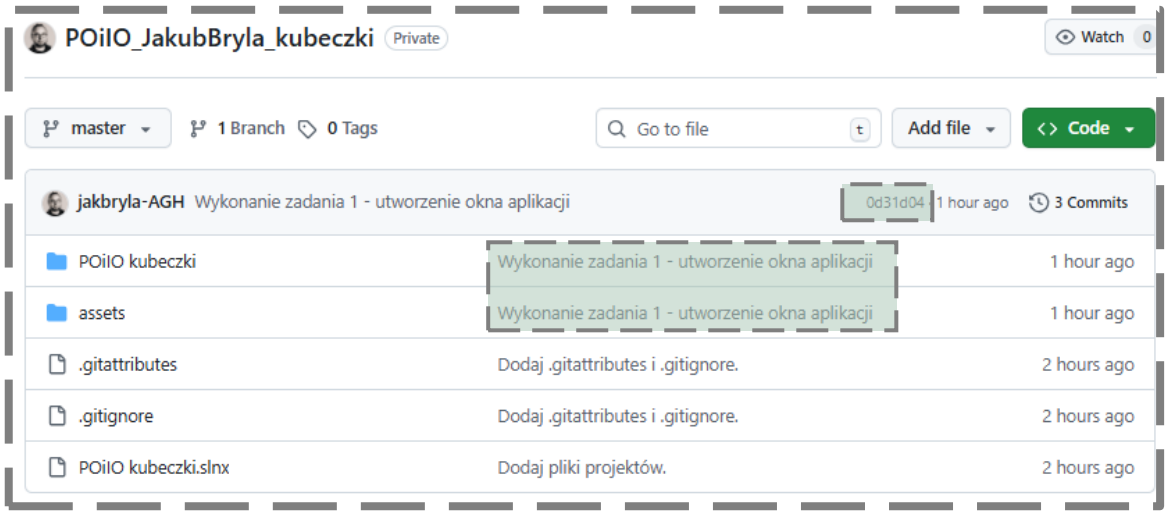
Kliknij znaczek „+” w celu dodania wszystkich zmian do poczekalni. Wprowadź komentarz „Wykonanie zadania 1 – utworzenie okna aplikacji” i Kliknij „Zatwierdź wszystko” – zostanie wykonany *commit*:



Pomyślność akcji można sprawdzić w konsoli:

```
C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki>git log --oneline
0d31d04 (HEAD -> master) Wykonanie zadania 1 - utworzenie okna aplikacji
609ed00 (origin/master, origin/HEAD) Dodaj pliki projektów.
5f31286 Dodaj .gitattributes i .gitignore.
```

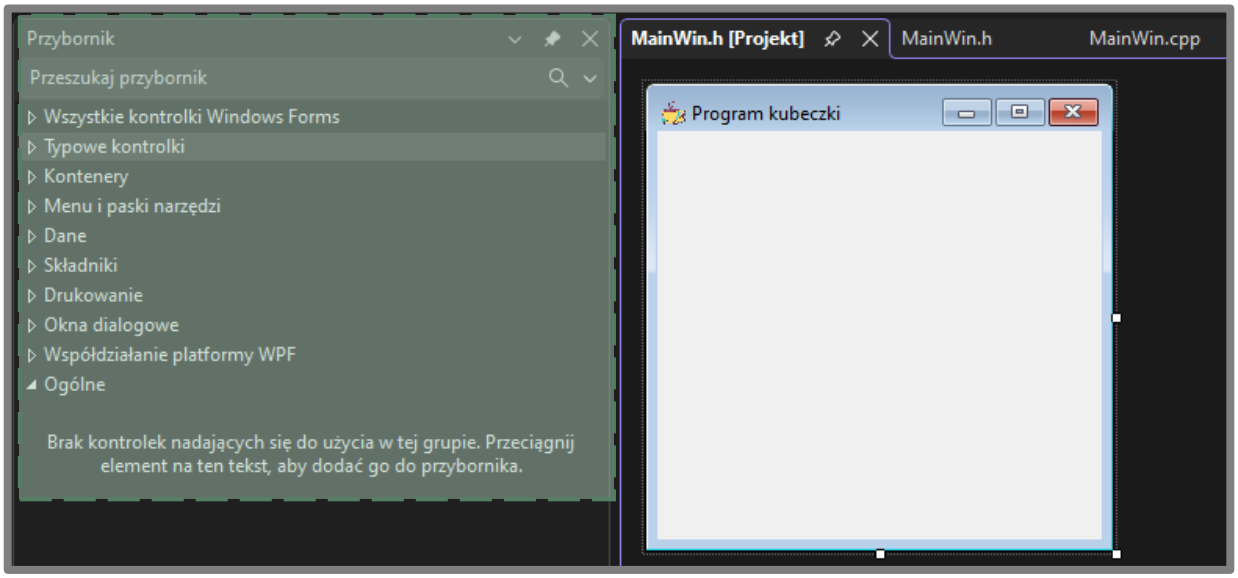
Pozostaje wysłać zmiany na repozytorium zdalne. W tym celu kliknij przycisk . Sprawdź zmiany w repozytorium:



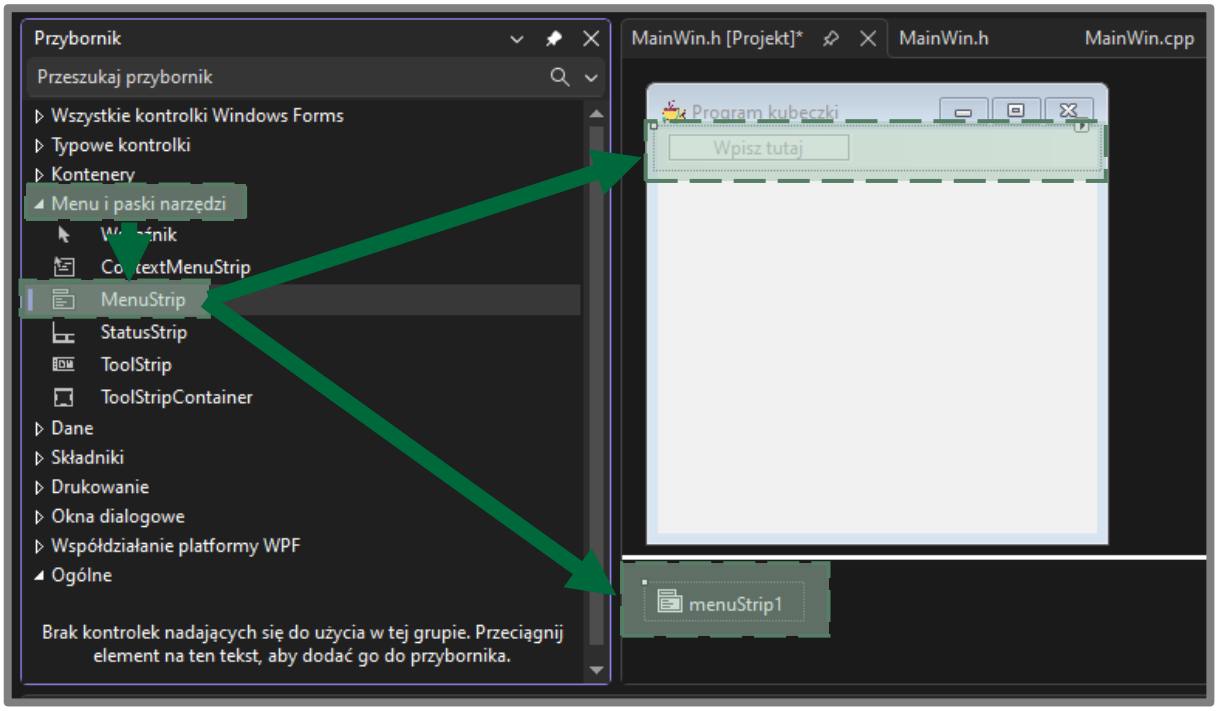
ZADANIE 4.2 – Utworzenie menu aplikacji

Niezależnie od tworzonej aplikacji zapewne będziesz potrzebować menu. W celu jego dodania do projektu otwórz w *Visual Studio* ***Przybornik***:

- w górnym pasku wybierz opcję Widok → Przybornik,
- LUB skorzystaj ze skrótu klawiszowego: „Ctrl+Alt+X”.



W ***Przyborniku*** wybierz zakładkę „*Menu i paski narzędzi*” i kliknij dwukrotnie lewym przyciskiem myszy na opcję „*MenuStrip*”:



Po wykonaniu akcji w głównym widoku IDE zobaczysz nowy komponent:

- w oknie aplikacji,
- w zakładce poniżej okna aplikacji.

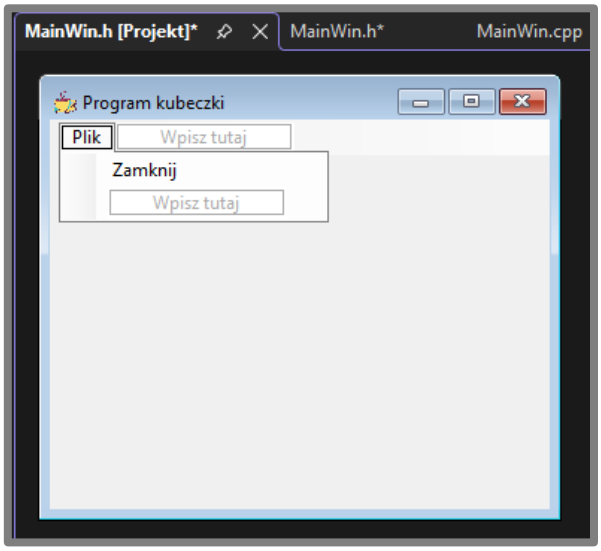
Co istotne, w pliku nagłówkowym *MainWin.h* został wygenerowany kod odpowiedzialny za powstanie nowego elementu GUI:

```
MainWin.h [Projekt]*   MainWin.h*   MainWin.cpp
POiIO kubeczki   POiIOkubeczki::MainWin
46      #pragma region Windows Form Designer generated code
47              /// <summary>
48              /// Metoda wymagana do obsługi projektanta - nie należy modyfikować
49              /// jej zawartości w edytorze kodu.
50              /// </summary>
51              void InitializeComponent(void)
52              {
53                  System::ComponentModel::ComponentResourceManager^ resources = (gcnew Sys
54                  this->menuStrip1 = (gcnew System::Windows::Forms::MenuStrip());
55                  this->SuspendLayout();
56                  //
57                  // menuStrip1
58                  //
59                  this->menuStrip1->Location = System::Drawing::Point(0, 0);
60                  this->menuStrip1->Name = L"menuStrip1";
61                  this->menuStrip1->Size = System::Drawing::Size(284, 24);
62                  this->menuStrip1->TabIndex = 0;
63                  this->menuStrip1->Text = L"menuStrip1";
64                  //
65                  // MainWin
66                  //
67                  this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
68                  this->AutoScaleMode = System::Windows::Forms::AutoScaleMode::Font;
69                  this->ClientSize = System::Drawing::Size(284, 261);
70                  this->Controls->Add(this->menuStrip1);
71                  this->Icon = (cli::safe_cast<System::Drawing::Icon^>(resources->GetObje
72                  this->MainMenuStrip = this->menuStrip1;
73                  this->Name = L"MainWin";
74                  this->Text = L"Program kubeczki";
75                  this->ResumeLayout(false);
76                  this->PerformLayout();
77
78              }
79      #pragma endregion
```

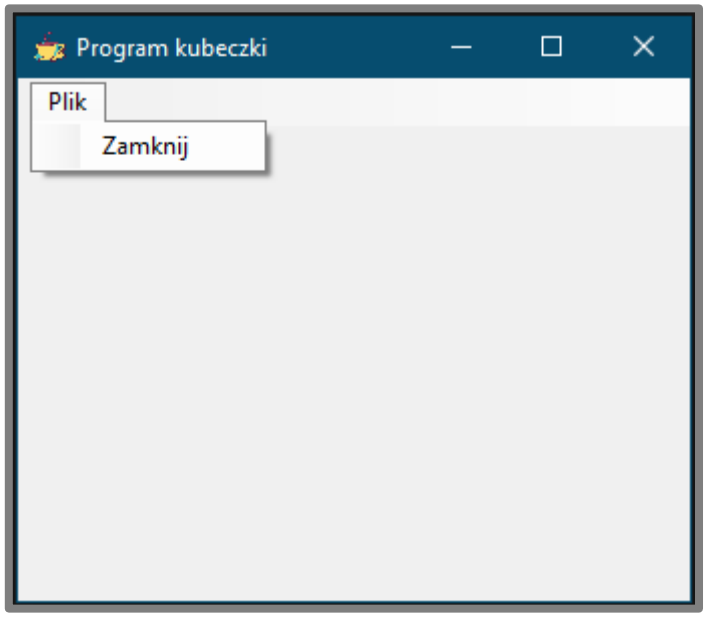
Nie ma potrzeby ciągłego śledzenia zmian w kodzie pliku nagłówkowego – do tej pory robiliśmy to dla celów poznawczych. Od tej pory nie będziemy śledzić każdej zmiany w pliku *MainWin.h*. Co więcej warto zwinąć fragmenty kodu automatycznie wygenerowane przez IDE:

```
MainWin.h [Projekt]*   MainWin.h*   MainWin.cpp
POiIO kubeczki   (Globalny zasięg)
1       #pragma once
2
3       namespace POiIOkubeczki {
4
5   >       using namespace ...
11
12  >       /// ...
15          public ref class MainWin : public System::Windows::Forms::Form
16          {
17          public:
18  >           MainWin(void) { ... }
25
26          protected:
27  >           /// ...
30  >           ~MainWin() { ... }
37          private: System::Windows::Forms::MenuStrip^ menuStrip1;
38          protected:
39
40          private:
41  >           /// ...
44              System::ComponentModel::Container ^components;
45
46  >       Windows Form Designer generated code
80          };
81      }
82
```

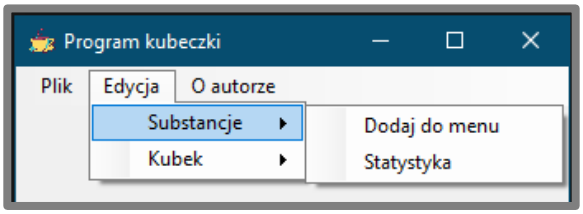
Przejdźmy z powrotem do widoku aplikacji – zakładki „*MainWin.h [Projekt]*”. Kliknij dwukrotnie w menu na okno „*Wpisz tutaj*” i utwórz pierwszą opcję wyboru „*Plik*”. Po zatwierdzeniu klawiszem ***Enter*** nowa zakładka zostanie utworzona i można stworzyć dla niej listę wyboru. Póki co ograniczymy się jedynie od opcji „Zamknij”. Efekt powyższych działań:

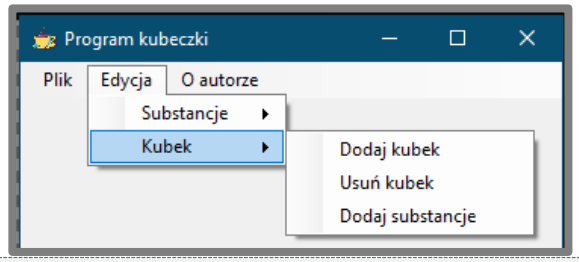


Po uruchomieniu programu:



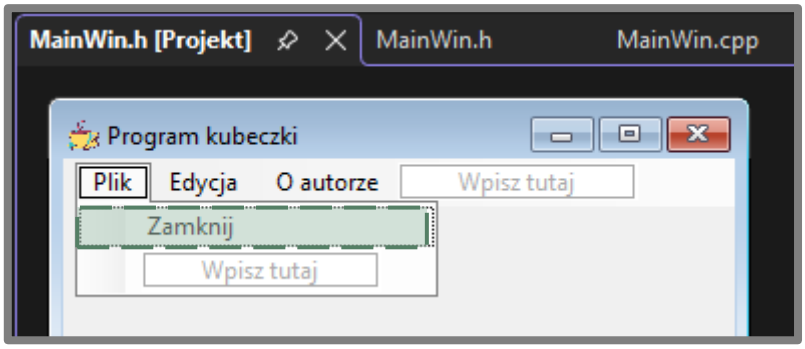
Utwórz kolejne opcje menu, aby uzyskać efekt zamieszczony na poniższym zrzucie ekranu:



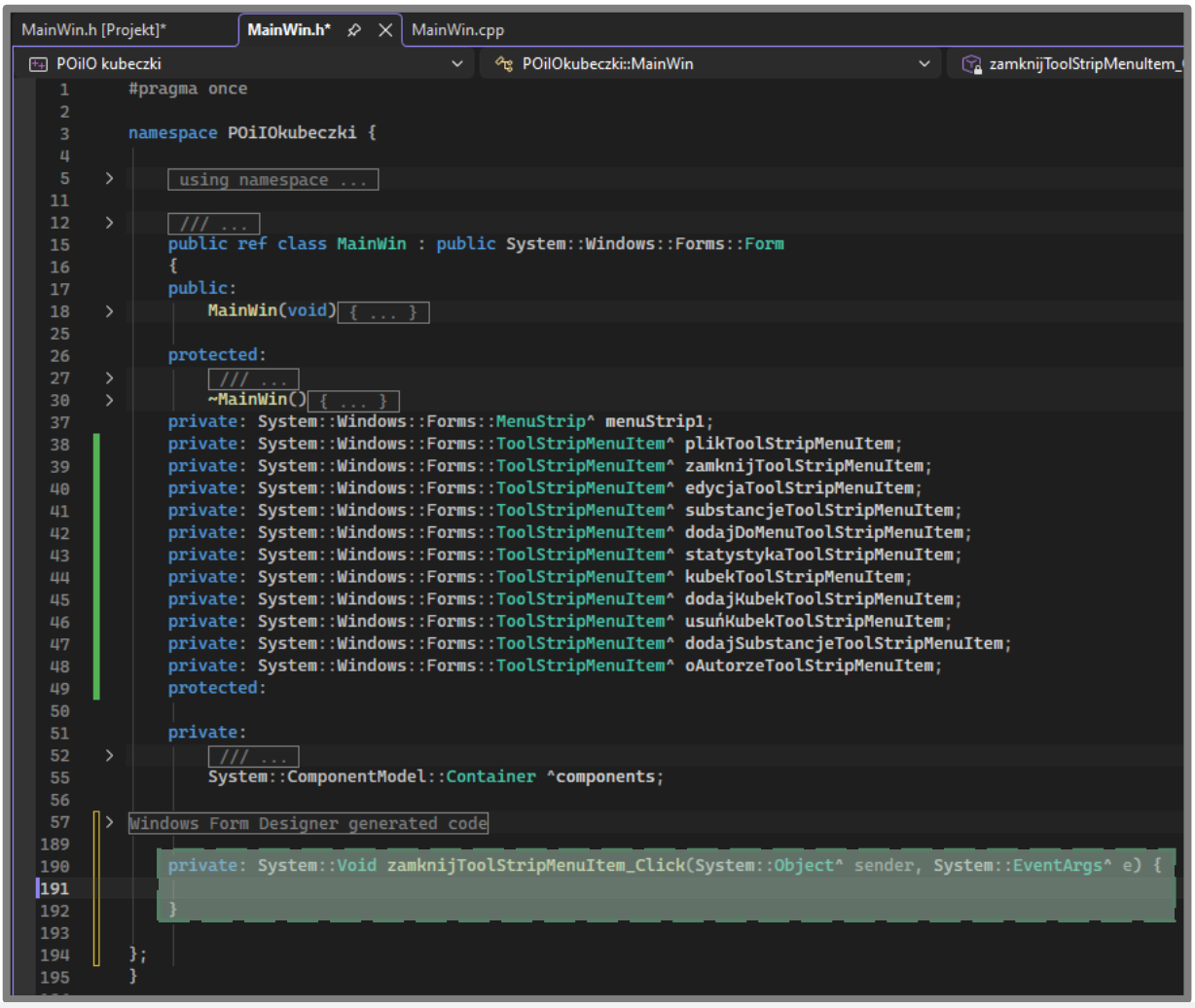


Zakładka „*O autorze*” nie posiada rozwijanej listy – kliknięcie na wspomnianą opcję spowoduje wyświetlenie nowego okna z danymi programisty. Tym zajmiemy się w kolejnym zadaniu. Podobnie, implementacją kodu dla zakładki „*Edycja*” poświęcone będą kolejne instrukcje.

Zajmijmy się kodem dla przycisku ***Zamknij*** w zakładce „*Plik*”. Kliknij dwukrotnie lewym przyciskiem myszy na wspomniany przycisk w widoku projektu.



W efekcie IDE przekieruje Cię do pliku MainWin.h, gdzie został wygenerowany szkielet nowej funkcji odpowiedzialnej za obsługę zdarzenia kliknięcia na przycisk:

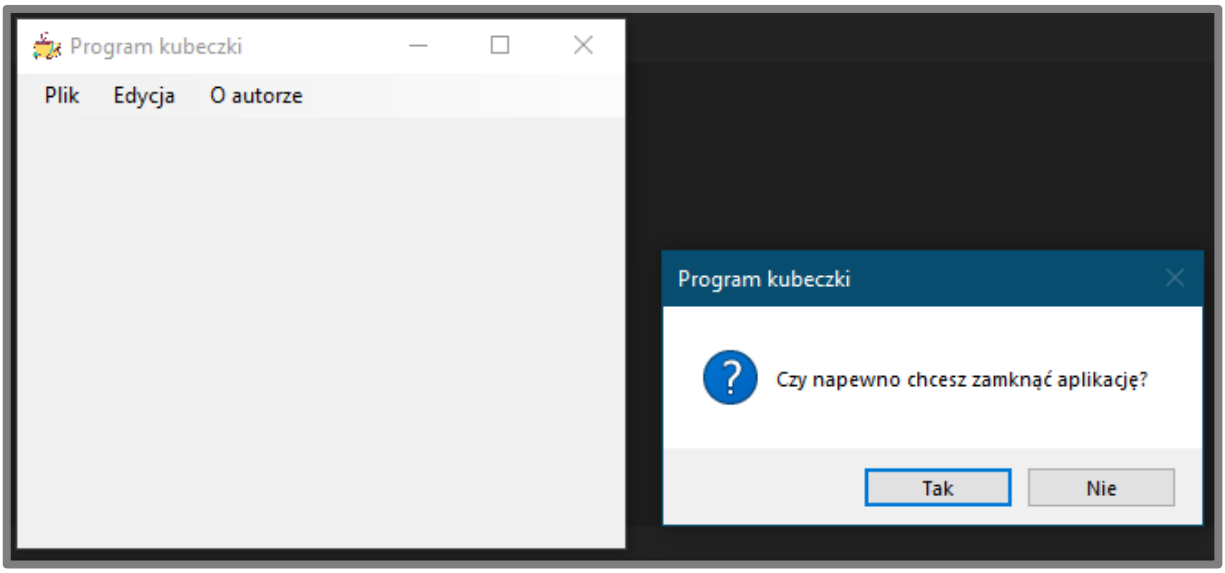


Podczas próby zamknięcia aplikacji dopytamy użytkownika czy jest tego pewny – pojawi się okno do zatwierdzenia decyzji. Dopiero kliknięcie przycisku ***Tak*** spowoduje zamknięcie aplikacji. Kod funkcji:

```
private: System::Void zamknijToolStripMenuItem_Click(System::Object^ sender, System::EventArgs^ e) {
        System::Windows::Forms::DialogResult answer;
        answer = MessageBox::Show("Czy napewno chcesz zamknąć aplikację?", "Program kubeczki", MessageBoxButtons::YesNo, MessageBoxIcon::Question);

        if (answer == System::Windows::Forms::DialogResult::Yes) Application::Exit();
}
```

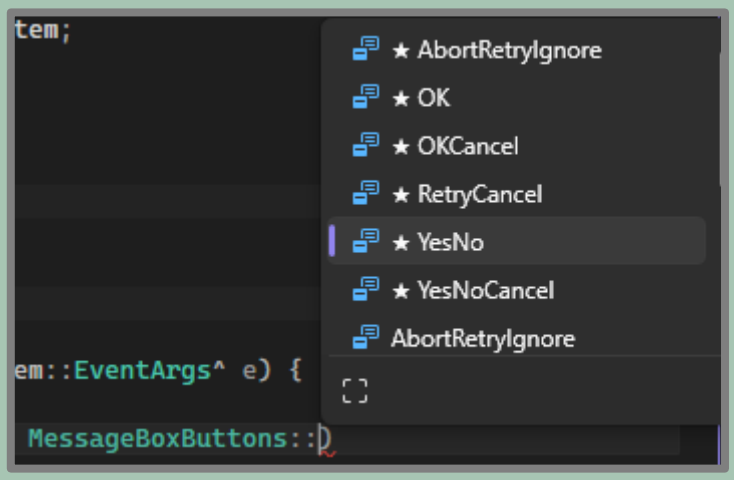
Skompiluj program i sprawdź obsługę przycisku ***Zamknij***:



Zwróć uwagę, że wewnątrz funkcji zostało wywołane utworzenie okna *MessageBox*, które przyjmuje 4 argumenty:

1. Treść zapytania: „Czy na pewno chcesz zamknąć aplikację?”,
2. Tytuł okna widoczny w jego górnym pasku: „Program kubeczki”,
3. Wybór dostępnych opcji odpowiedzi na zadane pytanie – w tym przypadku przyciski ***Tak*** i ***Nie***,

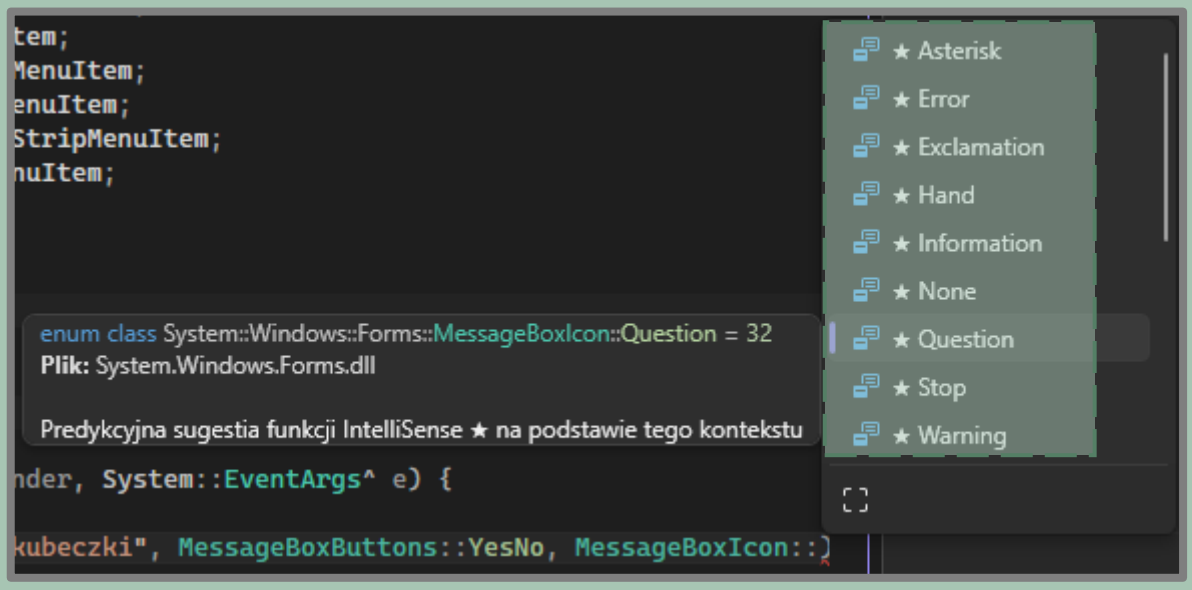
Podczas pisania kodu w IDE pojawiają się podpowiedzi odnośnie możliwych opcji:



W przypadku, gdyby okno odpowiedzi się nie pojawiło, możesz je wywołać klikając skrót klawiszowy „***Ctrl + J***”.

4. Ikonę wyświetlaną obok treści komunikatu – w tym przypadku znak pytajnika.

Podobnie jak powyżej, IDE podpowie jakie ikony można dodać do *MessageBox*:



Wybór użytkownika, kliknięcie na przycisk ***Tak*** lub ***Nie***, zapisuje do zmiennej pomocniczej *answer*.

Skąd wiem jakiego typu jest zmienna *answer*? Skorzystałem tu z podpowiedzi… Początkowo utworzyłem kod poniżej, aby podejrzeć zwracany typ przez *MessageBox*:

```
auto a = MessageBox::Show("Czy napewno chcesz zamknąć aplikację?", "P
    (zmienna lokalna) System::Windows::Forms::DialogResult a
    System::Windows::Forms::
    Wyszukaj w trybie online
```

Finalnie zrezygnowałem z automatycznego wykrywania typu zmiennej, tak aby kod był bardziej zrozumiały/przejrzysty – analizując powtórnie kod widzę jakiego konkretnie typu są zmienne.

Ostatecznie, w instrukcji warunkowej sprawdzam czy kliknięty został przycisk ***Tak***. W takim przypadku zamykam aplikację poleceniem `Application::Exit();`.

Wykonaj commit do repozytorium lokalnego z komentarzem: „Wykonanie zadania 2 – zamykanie aplikacji z menu”. Prześlij zmiany do repozytorium zdalnego.

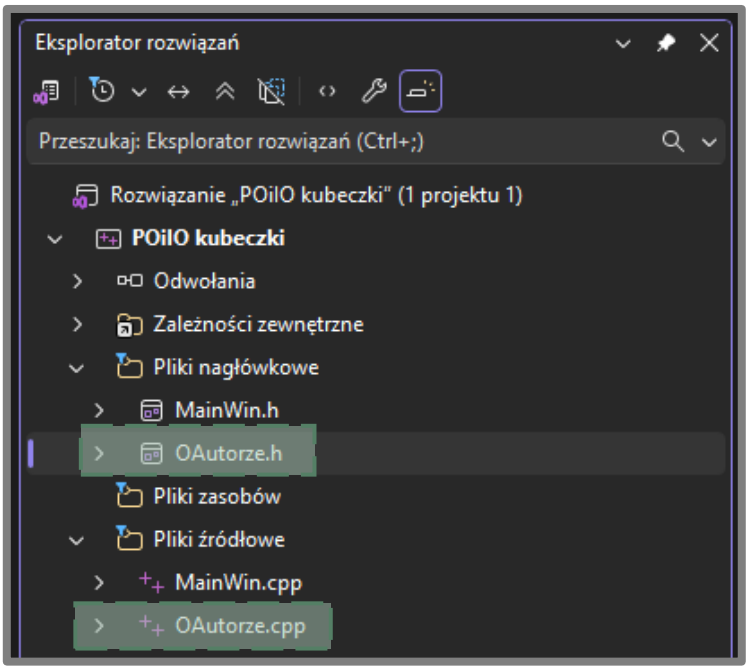
```
C:\Users\jakbr\Documents\Cpp_projects\POiIO kubeczki>git log --oneline
4679d4d (HEAD -> master, origin/master, origin/HEAD) Wykonanie zadania 2 - zamykanie aplikacji z menu
0d31d04 Wykonanie zadania 1 - utworzenie okna aplikacji
609ed00 Dodaj pliki projektów.
5f31286 Dodaj .gitattributes i .gitignore.
```

jakbryla-AGH Wykonanie zadania 2 - zamykanie aplikacji z menu		4679d4d · 2 minutes ago	4 Commits
POiIO kubeczki	Wykonanie zadania 2 - zamykanie aplikacji z menu	2 minutes ago	
assets	Wykonanie zadania 1 - utworzenie okna aplikacji	1 hour ago	
.gitattributes	Dodaj .gitattributes i .gitignore.	2 hours ago	
.gitignore	Dodaj .gitattributes i .gitignore.	2 hours ago	
POiIO kubeczki.slnx	Dodaj pliki projektów.	2 hours ago	

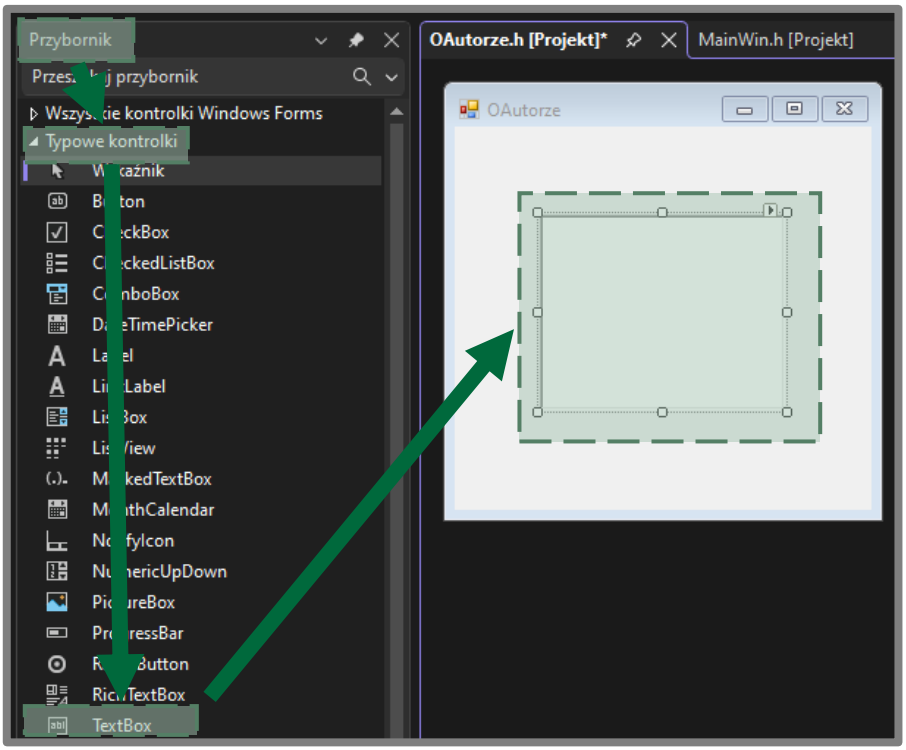
ZADANIE 4.3 – wyświetlenie informacji o autorze w nowym oknie aplikacji

W celu wyświetlenia informacji o autorze konieczne jest utworzenie nowej *formy*. Kliknij prawym przyciskiem myszy na nazwę projektu w *Eksploratorze rozwiązań* i wybierz opcję *Dodaj* → *Nowy element*. Przejdź do zakładki *Interfejs użytkownika*, wybierz *Formularz systemu Windows*, wprowadź nazwę „*OAutorze*" i kliknij przycisk *Dodaj*.

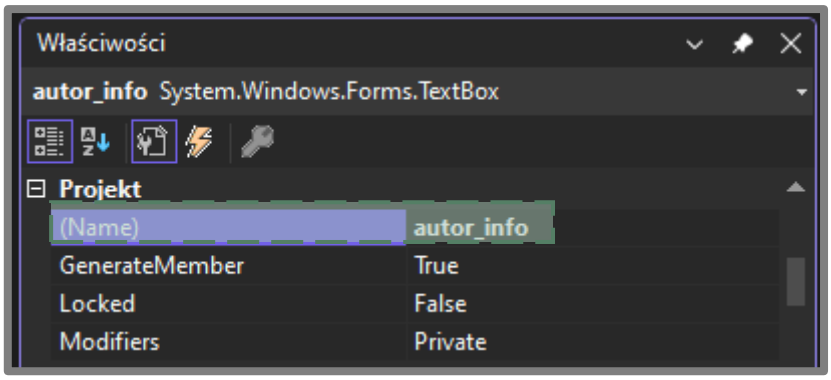
W efekcie w IDE powinna otworzyć się nowa zakładka „*OAutorze.h [Projekt]*", a w *Eksploratorze rozwiązań* powinny pojawić się 2 nowe pliki:



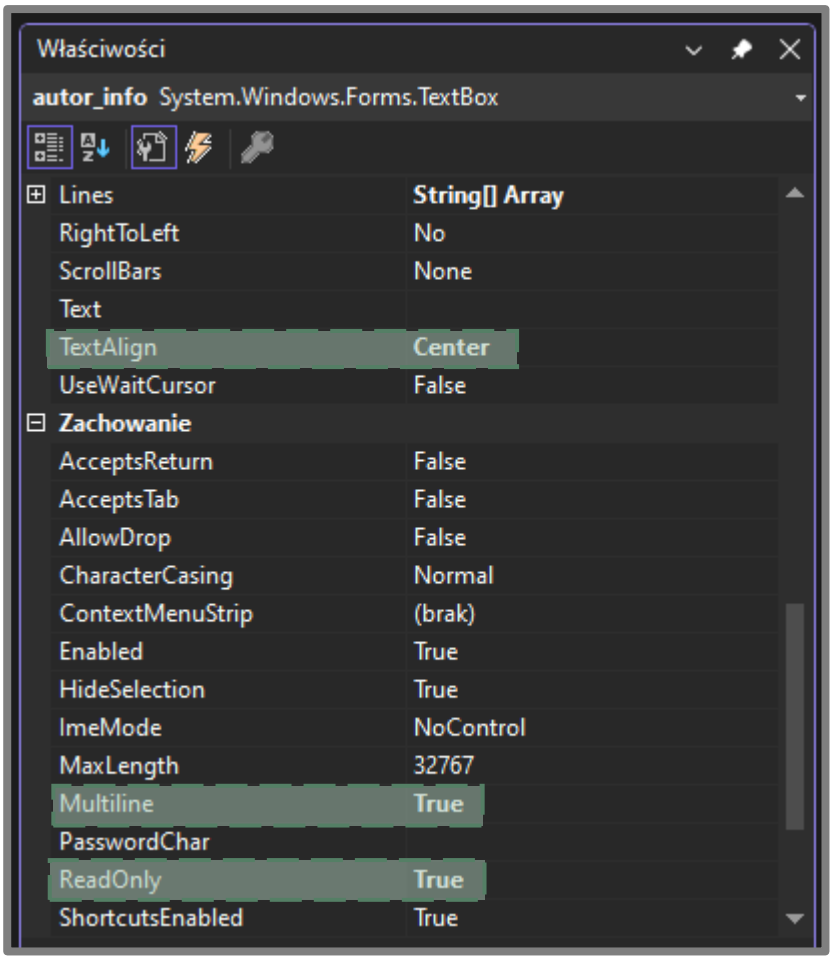
Z ***Przybornika*** wybierz zakładkę „*Typowe kontrolki*", następnie T*extBox* i umieść go po środku okna aplikacji:



W oknie ***Właściwości*** zmień nazwę kontrolki:



Następnie ustaw justowanie tekstu, tekst wieloliniowy oraz brak możliwości edycji tekstu:



Kliknij dwukrotnie lewym przyciskiem myszy na pusty obszar okna aplikacji (poza kontrolką). Zostanie wygenerowany szkielet funkcji automatycznej wywoływanej w trakcie otwierania okna. Wstaw kod funkcji:

```
private: System::Void OAutorze_Load(System::Object^ sender, System::EventArgs^ e) {
        autor_info->Text = "\r\n\r\nProgram kubeczki\r\n=====\r\nAutor: dr inż. Jakub Bryła\r\n=====\r\n2026";
}
```

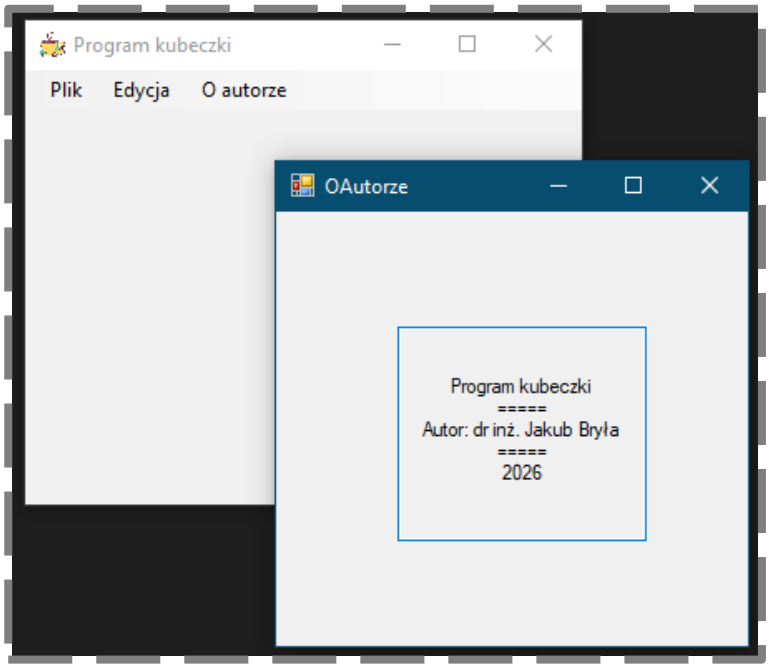
Przejdź do zakładki „*MainWin.h [Projekt]*” i kliknij dwukrotnie lewym przyciskiem myszy na opcję menu „*O Autorze*”. Zostanie wygenerowany szkielet funkcji, do której należy wstawić kod:

```
1  OAutorze^ autor_info = gcnew OAutorze();
2  autor_info->Show();
```

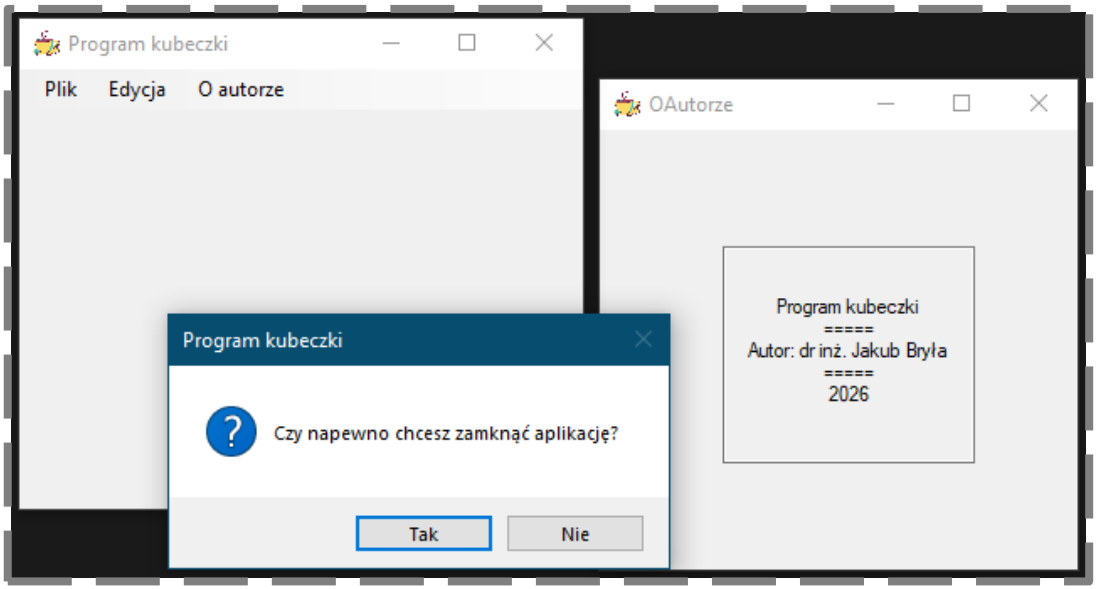
Ostatecznie, w drugiej linijce pliku *MainWin.h*, za instrukcją `#pragma once`, należy dodać referencje do *OAutorze.h*:

```
1  #include "OAutorze.h"
```

Efekt powyższych działań, po uruchomieniu programu i kliknięciu opcji „O autorze”:



Możesz również dodać ikonę aplikacji do nowego okna. Finalny efekt:



Zaktualizuj stan repozytorium lokalnego i zdalnego.

5. Podsumowanie

Instrukcja stanowi wprowadzenie do tworzenia aplikacji okienkowych w języku C++/CLI. Utworzyłeś szkielet dla nowego projektu, skupiając się na stworzeniu głównego okna programu oraz jego menu. Nauczyłeś się korzystać z *MessageBox*, obsługi wyboru użytkownika oraz tworzenia okna pomocniczego programu – z informacjami o autorze. Są to ogólne funkcjonalności spotykane w większości aplikacji. W kolejnych instrukcjach skupimy się na często używanych kontrolkach oraz ich obsłudze w kontekście projektu kubeczków.